

Squall

Squall (*student proposal*)

Assessment: ✓ Fit · **Category:** Games · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval

Proposal (as submitted): *Squall* is a **physics-based puzzle game** where the instrument of manipulation is the player's fingers: players **draw push, pull and time-freeze spells directly onto the screen** as shapes to manipulate the world — jamming a mechanism with a box, yanking a key from across an inaccessible area, freezing objects to accelerate them toward a locked gate. Every puzzle has a goal but the solution is in the player's hands; three simple base spells let level architecture, systems and introduced mechanics carry the challenge. Named load-bearing C++: a stable **rigid-body physics loop** running many puzzle objects simultaneously; an **ECS** (contiguous memory for cache-efficient updates, decoupled rendering / physics / state-evaluation systems, transient event components + buffers for interaction handling); and a **raycasted audio service** casting dynamic sound rays for realistic acoustic occlusion + distance. Named technical features: **vector/direction gesture recognition** for drawn spells (real-time geometry + direction analysis distinguishing a circle from another shape); the dynamic physics system; the raycasted audio; and a **level-streaming database** for instant load (stream assets instead of loading all upfront). Tools: a **level editor** (free room layout — collision walls, spell/object-gated paths, room environment effects; entry/exit transition points) and a **scriptable object system** (transform/rotation/scale/ID-tag properties; typed objects like light sources, switches, buttons, pressure plates; custom script data for object-to-object interaction; reusable data assets placed many times). Design features: **location-sensitive screen-drawing** of spells; physics-based environmental interactions; the **three spells** — a **circle** that pulls surrounding objects, a **triangle** that freezes objects in time (accumulated forces/status effects all resolve on elapse), and a **push** that forces objects along/in front of it; and **non-combative boss fights** solved as puzzles using in-world objects. Team: **6 CS + 1 Design + 1 Art**.

Note (module completeness): the proposal doesn't name a **platform** — the screen-drawn gesture spells are touch-first, so this is planned as a mobile build (Pixel 9a per the module). It also doesn't mention an **External Database** or accounts, but the module mandates Firebase — plan to persist level-completion / progress / settings there (a standard, small addition; see the External Database row + M1G12-style integration). Neither is a category concern.

References:

- [Crayon Physics Deluxe on Steam](#) (a 2D physics puzzle where the player's drawings become real physical objects that solve the puzzle — the draw-a-shape-to-manipulate-physics precedent)
- [Ōkami — Celestial Brush \(Ōkami Wiki\)](#) (the player pauses the world and draws shapes/strokes on screen, each recognised gesture triggering a distinct world-manipulation effect — the screen-drawn-gesture → recognised-effect precedent)
- [World of Goo on Steam](#) (a physics-based puzzle where a small toolkit and level architecture — not raw power — create the challenge, and the player chooses the solution — the physics-puzzle-from-simple-rules precedent)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

Core Tech Requirement	Squall
External Database	✓
Authentication »	●
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	✓
3D (strong students only)	✕
Networking System	●

How each requirement applies:

Platform fit: **Main:** Native mobile (Native Android — C++ engine via JNI rendering through OpenGL ES in a GLSurfaceView, Kotlin app shell; device: Pixel 9a) — the screen-drawn spells are touch-first; a Web-on-mobile (Emscripten→WASM) build is a viable alternative. The Native PC build is the development/tools vehicle (the level editor + scriptable-object system).
Extended: Firebase + a Linux server the team sets up by M2 (full use T2) for progress / level-completion persistence.

Front end: A 2D physics-puzzle renderer — the rooms, the many rigid-body objects (boxes, keys, gates, switches, pressure plates), the live gesture stroke

the player draws, the three spell effects, and the puzzle/goal HUD.

Languages: Native — Kotlin Native Android UI + the C++ engine via JNI (Android Studio); OpenGL ES for the realtime 2D in a GLSurfaceView; Web/desktop — SDL2/3 + OpenGL ES for the PC dev/tools build.

Back end: The real-time C++ engine — the rigid-body physics loop over many puzzle objects, the ECS (contiguous storage, decoupled render/physics/state systems, transient event components), the real-time vector/direction gesture-recognition service, the raycasted audio service (occlusion + distance), and the level-streaming system; plus Firebase for progress persistence. **Languages:** C++ (engine + services); Firebase-backed services on the Extended tier.

Backend scope: Local now, server-ready (T2) — the progress-persistence seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* three load-bearing C++ pieces — the stable rigid-body physics loop (many simultaneous objects, the freeze-then-release-forces mechanic), the ECS (cache-efficient contiguous storage, decoupled systems, event buffers), and the raycasted audio service (dynamic sound rays for occlusion/distance); the gesture recognizer analyses stroke geometry + direction in real time to classify the spell.
- *Data-driven:* object types + properties (transform/scale/ID, light sources, switches, buttons, pressure plates), the three spell definitions, room layouts, spell/object path-gates and environment effects are all data — a new object type or puzzle is a data change (authored via the editor + scriptable-object system).
- *2D:* the puzzle-room renderer, the live drawn-gesture stroke, the three spell VFX and the goal HUD (custom-rendered). *3D:* not needed.
- *Custom tools:* the **level editor** (room layout — collision walls, spell/object-gated paths, environment effects, entry/exit transitions) and the **scriptable object system** (typed, property-carrying, reusable data assets with object-to-object scripts).
- *Networking:* single-player; progress persistence is async client↔cloud (Application Communication ✓), so a Networking System is optional (●).
- *External DB:* level-completion, progress and settings (Firebase — add this per the module). *Auth* »: optional accounts. *Analytics* »: optional — per-puzzle solve-time + spell usage for balancing.

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Games pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic** — the ECS loop + the real-time **gesture-recognition** input service.

Tech & dependencies

- **Tech: Web build:** SDL2/3 window + a fixed-timestep C++ loop over an ECS (contiguous storage, decoupled render/physics/state systems, transient event components/buffers), with a real-time vector/direction gesture recognizer that classifies the drawn stroke (circle/triangle/line) into a spell; C++17/20, CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; the physics (M1G5) and rendering (M1G4) run on its tick; the spells consume the recognizer.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** An ECS/entity model with object types + properties (transform/scale/ID; light sources, switches, buttons, pressure plates), the three spell definitions, room layouts and path-gates in hot-reloadable JSON — a new object type or puzzle is a data change; the basis for the scriptable-object system.
- **Prerequisites:** G1 (loop & entity hooks).
- **Feeds into:** M2G2 (level editor), M2G6 (scriptable-object system), M1G5 (physics bodies from object data).

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui overlays (physics bodies, recognized-gesture confidence, event-buffer state), Tracy profiling of a room with many rigid bodies + simultaneous spells to prove the stable step, Catch2 tests (gesture

classification, physics determinism, data loading), on a GitHub Actions Windows+Android matrix.

- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G3 (CI hardening); the physics budget M2G1 validates.

- **G4 2D Graphics** — the puzzle-room renderer + the live drawn-gesture stroke + the three spell VFX.

Tech & dependencies

- **Tech: Web build:** A sprite/shape renderer for rooms and the many physics objects, a camera, the live gesture-stroke overlay, the pull/push/freeze spell visuals and the goal HUD; SDL2/3/OpenGL ES, GLSL, stb_image, FreeType. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G1 (loop & window).
- **Feeds into:** M2G9 (raycasted audio pairs with the visual scene); the level editor shares this renderer.

- **G5 Physics & Collision** — the stable rigid-body loop for many simultaneous puzzle objects.

Tech & dependencies

- **Tech:** A C++ rigid-body integrator + broadphase (uniform grid / quadtree) running many bodies stably, with forces/impulses that the pull/push/freeze spells apply; deterministic enough for reproducible puzzle solutions; GLM math.
- **Prerequisites:** G1 (loop & tick), G2 (object data).
- **Feeds into:** M2G11 (advanced physics — the freeze-then-release-forces mechanic, constraints); every spell interaction.

- **G14 Level / Tile Map System** — the **level-streaming** database for instant load.

Tech & dependencies

- **Tech:** A chunked/streamed level format (JSON/asset records) with collision data and spell/object path-gates, streaming rooms in/out around transitions so nothing loads all-upfront; spatial indexing; the data format the editor authors.
- **Prerequisites:** M1G2 (object/room data model).

- **Feeds into:** M2G2 (the level editor writes this format); smooth room transitions; instant-load at showcase.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation** — the physics + streaming budget hardened + on-device validation.

Tech & dependencies

- **Tech: Web build:** Memory/component pools, cache-friendly ECS iteration, Tracy profiling of a many-body room with simultaneous spells, ASan on MSVC, and on-device validation (Pixel 9a) holding the frame budget with streaming; the Firebase/Linux-server seam (progress persistence) set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** All M1 goals (hardens the M1 codebase).
- **Feeds into:** The mobile deliverable + a stable step under many bodies.

- **G2 ★ Content Editor — Core (Level Editor)** — room layout: collision walls, spell/object-gated paths, environment effects, entry/exit transitions.

Tech & dependencies

- **Tech:** Dear ImGui + the shared renderer: place walls, paths and path-gates (spell/object filters), room environment effects and entry/exit transition points, with round-trip JSON to the streaming format, live preview and undo/redo.
- **Libraries/Software:** Dear ImGui, nlohmann/json, the project's renderer (preview).
- **Prerequisites:** G2 (object/room data), M1G14 (streaming format).
- **Feeds into:** All level content; the custom-tools showcase.

- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, reproducible Windows + Android builds, tests (gesture classification, physics determinism, streaming/data loading) and packaging via GitHub Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView

where realtime 2D is needed — no SDL on native, some screens are pure Native UI.

- **Prerequisites:** G3 (CI & test foundation).
- **Feeds into:** Release/showcase builds of every goal.

- **G6 Content Editor — Advanced Tool Features (Scriptable Object System)** — typed, property-carrying, reusable objects with object-to-object scripts.

Tech & dependencies

- **Tech:** Dear ImGui advanced widgets: define object types (light source, switch, button, pressure plate) with transform/rotation/scale/ID and custom script data linking objects to each other, saved as reusable data assets placed many times; drag-drop, live preview binding, undo/redo depth.
- **Prerequisites:** M2G2 (editor core), G2 (data model).
- **Feeds into:** Fast puzzle authoring; the content showcase.

- **G9 Audio — Advanced (Raycasted Audio Service)** — dynamic sound rays for acoustic occlusion + distance.

Tech & dependencies

- **Tech:** FMOD/OpenAL + a C++ service casting sound rays across the scene geometry to compute occlusion and distance attenuation in real time, with spatialization and mixing buses; data-driven emitter definitions.
- **Prerequisites:** M1 audio hooks; M1G5 (scene geometry to cast against).
- **Feeds into:** The atmospheric-design pillar; showcase audio polish.

- **G11 Physics — Advanced (Freeze-Force Mechanic & Constraints)** — the triangle freeze-then-release, constraints, continuous collision.

Tech & dependencies

- **Tech:** Constraint/joint solvers, continuous collision, and a time-freeze mechanic that accumulates forces/status effects on frozen bodies and resolves them all on elapse; materials for consistent feel; GLM math, deterministic for reproducible solutions.
- **Prerequisites:** M1G5 (physics & collision).
- **Feeds into:** The signature freeze spell; the non-combative puzzle-boss interactions.

Track Goals & Team Composition

Recommended composition: 6 CS + 1 Design + 1 Art (8) — **as proposed**, and well-matched: a physics-puzzle with a small object/spell vocabulary is a light-but-real art surface (readable puzzle objects, the three spell VFX, room environments, UI), which one artist can carry, while a single designer owns the puzzle/mechanic design, the level architecture and the spell-interaction systems (rules: min 1 Design mandated, max 9 total, $D+A \leq 3$; see [team-formation.pdf](#)). If the puzzle-content volume or the scriptable-systems design outgrows one designer, consider trading toward a 2nd designer. *Scope note (advisory):* a stable many-body physics loop + real-time gesture recognition + a raycasted-audio service + level streaming + a level editor + a scriptable-object system is a **CS-heavy, ambitious** scope — the three spells acting on a rigid-body room, gesture recognition for the three shapes, and a handful of hand-built puzzles is the M1/M2 floor; the raycasted audio, the streaming database, the full scriptable-object editor and the non-combative puzzle-bosses are the stretch. Recommend picking **one** of raycasted-audio *or* level-streaming as the M2 "hero" system if time is tight (both are genuinely optional-polish relative to the core physics+gesture loop).

Track goals — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the three-spell physics-puzzle USP, the location-sensitive drawing, the puzzle-goal-but-open-solution philosophy and the non-combative puzzle-bosses; primary benchmark e.g. Crayon Physics / World of Goo, plus a Singapore benchmark)
- M1D2 Core Gameplay Loop Prototype (a greybox proving draw → recognise → physics-effect → solve is satisfying with the three spells) · prototype in Unity
- M1D8 Puzzle & Mechanic Design (the three spells' interaction catalogue, a difficulty ramp, and 5–8 designed puzzles authored to the CS data format)

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D2 Working Prototype — Full-Content Build (the polished puzzle set + a non-combative boss, in the team's engine + editor)

- **UI/UX M1** (graded by the Design instructor for the Games category):
 - M1U1 ★ UI Standards & Wireframe Baseline — readability standards for the spell-draw affordance, the goal/objective display and the puzzle-state feedback.
- **UI/UX M2:**
 - M2U1 ★ Functional UI & Usability — menus, level select, the in-puzzle HUD and the spell-draw feedback navigable unaided on the mobile target.
 - M2U2 ★ UX Validation & Final Polished UI — validated, polished draw-spell + puzzle UX tested with players on the Pixel 9a.
- **Art M1:** 1 artist (puzzle objects + spell VFX + rooms + UI)
 - M1A1 ★ Art Bible / Proof of Concept (the visual direction — palette, tone, the object/spell readability language)
 - M1A3 Environment, Props & VFX Concepts (the puzzle rooms, the interactable objects, and the pull/push/freeze spell-effect concepts)
 - M1A4 UI Art Concepts & Typography (the spell-draw affordance, goal display, HUD)
- **Art M2:**
 - M2A1 ★ Critical Asset Production & In-Engine Visual Quality
 - M2A3 Environment Alive — Ambient Animation & Layering (living, atmospheric puzzle rooms that pair with the raycasted audio)
 - M2A5 VFX — Critical Effects & Transitions (the pull/push/freeze spell effects + room transitions)
- **TEAM M1:**
 - M1TA1 ★ Overall Audio Experience — initial BGM/SFX from the week-5 needs list present & working in the prototype (atmospheric BGM, spell-cast, pull/push/freeze, object-collision and puzzle-solved cues), suitable and correctly triggered.
 - M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback (the base the raycasted-audio service extends).
- **TEAM M2:**
 - M2TA1 ★ Overall Audio Experience — standard Games floor (atmospheric BGM + spell / physics-impact / freeze / solve /

transition SFX, ≥ 6 types), integrated with the raycasted occlusion/distance model.

- M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel mixing; distinct per-spell cues).
- M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the level editor + the scriptable-object system), prototype demo, team post-mortem (wk14)

*Generated by the Project Proposal Assessor skill · grounding: live folder
(Project 2 Technical adjustments/) · references verified 2026-09-21.*